

Знакомство с Intel[®] oneAPI (oneTBB) часть 2

oneTBB parallel_for_each

parallel_for_each позволяет обрабатываться разными потоками пространство контейнера.

Заголовок `#include <tbb/parallel_for_each.h>`

```
template<typename InputIterator, typename Body>
void parallel_for_each(InputIterator first, InputIterator last,
    Body body);
```

```
template<typename InputIterator, typename Body>
void parallel_for_each(InputIterator first, InputIterator last,
    Body body, task_group_context& group);
```

```
template<typename Container, typename Body>
void parallel_for_each(Container& c, Body body);
template<typename Container, typename Body>
void parallel_for_each(Container& c, Body body,
    task_group_context& group);
```

```
template<typename Container, typename Body>
void parallel_for_each(const Container& c, Body body);
template<typename Container, typename Body>
void parallel_for_each(const Container& c, Body body,
    task_group_context& group);
```

oneTBB parallel_for_each

Пример обработки контейнер list

```
void test_parallel_for_each()
{
    std::list<int> mylist={ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

    int sum = 0;

    auto sum_up = [&sum](auto& value)
    {
        sum += value;
    };

    tbb::parallel_for_each(mylist.begin(), mylist.end(), sum_up);

    std::cout << "sum from 1 to 10 equals " << sum << std::endl;
}
```

oneTBB parallel_scan

Шаблон функции **parallel_scan** вычисляет префикс параллельного сканирования, также известный как параллельное сканирование.

Заголовок `#include <tbb/parallel_scan.h>`

```
template<typename Range, typename Body>
void parallel_scan(const Range& range, Body& body);
template<typename Range, typename Body>
void parallel_scan(const Range& range, Body& body, partitioner);

template<typename Range, typename Value, typename Scan, typename
Combine>
Value parallel_scan(const Range& range, const Value& identity,
const Scan& scan, const Combine& combine);
template<typename Range, typename Value, typename Scan, typename
Combine>
Value parallel_scan(const Range& range, const Value& identity,
const Scan& scan, const Combine& combine, partitioner);
```

oneTBV parallel_scan

Математическое определение параллельного префикса следующее.

Пусть $\blacksquare$ — ассоциативная операция с левым единичным элементом $\text{id}_{\blacksquare}$.

Параллельным префиксом $\blacksquare$ для последовательности $z_0, z_1, \dots, z_{n-1}$

будет последовательность $y_0, y_1, y_2, \dots, y_{n-1}$, где:

$$y_0 = \text{id}_{\blacksquare} \blacksquare z_0$$

$$y_i = y_{i-1} \blacksquare z_i$$

Например, если $\blacksquare$ — сложение, то параллельный префикс соответствует текущей сумме. Последовательная реализация параллельного префикса:

```
T temp = id;
for (int i = 1; i <= n; ++i)
{
    temp = temp + z[i];
    y[i] = temp;
}
```

Параллельный префикс выполнит код параллельно, повторно связывая применение $\blacksquare$ (+ в примере) за два прохода. $\blacksquare$ может вызываться в два раза больше по сравнению с вычислением последовательного префикса.

oneTBB parallel_scan

Пример императивная форма (объекта Body)

```
class Body {
    T sum;
    T* const y;
    const T* const z;
public:
    Body(T *y_, const T *z_) : sum(id), z(z_), y(y_) {}
    T get_sum() const { return sum; }

template<typename Tag>
void operator()(const tbb::blocked_range<int>& r, Tag)
{
    T temp = sum;
    int rend = r.end();
    for (int i = r.begin(); i < rend; ++i) {
        temp = temp + z[i];
        if (Tag::is_final_scan())
            y[i] = temp;
    }
    sum = temp;
}
```

oneTBB parallel_scan

Пример императивная форма (объекта Body) продолжение

...

```
Body(Body& b, tbb::split) : z(b.z), y(b.y), sum(id) {}
```

```
void reverse_join(Body& a) { sum = a.sum + sum; }
```

```
void assign(Body& b) { sum = b.sum; }
```

```
};
```

```
T DoParallelScan(T *y, const T *z, int n)
{
```

```
    Body body(y, z);
```

```
    tbb::parallel_scan(tbb::blocked_range<int>(0, n), body);
```

```
    return body.get_sum();
```

```
}
```

oneTBB parallel_scan

Пример на основе лямбда выражения

```
T DoParallelScan(T *y, const T *z, int n) {  
    return tbb::parallel_scan(  
        tbb::blocked_range<int>(0, n),  
        id,  
        [&](const tbb::blocked_range<int>& r, T sum, bool  
            is_final_scan)->T  
        {  
            T temp = sum;  
            int rend = r.end();  
            for (int i = r.begin(); i < rend; ++i) {  
                temp = temp + z[i];  
                if (is_final_scan)  
                    y[i] = temp;  
            }  
            return temp;  
        },  
        [&](T left, T right) {  
            return left + right;  
        }  
    );  
}
```


Выделение памяти

oneAPI Threading Building Blocks предоставляет шаблоны распределителя памяти: `scalable_allocator <T>` и `cache_aligned_allocator <T>`, решающие критические проблемы параллельного программирования :

- ❑ **Масштабируемость (Scalability).** Проблемы масштабируемости возникают при использовании распределителей памяти, изначально разработанных для последовательных программ, в потоках, которым, возможно, придется конкурировать за один общий пул таким образом, чтобы одновременно можно было выделить только один поток. **`scalable_allocator <T>`** позволяет избежать проблем с масштабируемостью. (может повысить производительность программ, которые быстро выделяют и освобождают память).
- ❑ **Ложный обмен (False sharing).** Проблема совместного использования строки кэша разными потоками. Проблема в том, что строка кэша - это единица обмена информацией между кешами процессора. Если один процессор изменяет строку кэша, а другой процессор читает ту же строку кэша, строка должна быть перемещена с одного процессора на другой, даже если два процессора имеют дело с разными словами в строке. Ложное совместное использование может снизить производительность, поскольку для перемещения строк кэша могут потребоваться сотни тактов. Шаблон **`cache_aligned_allocator<T>`** следует использовать, чтобы всегда выделять память в отдельной строке кэша.

Пример использования `cache_aligned_allocator<T>`,

Можно использовать шаблоны выделения памяти в качестве аргумента аллокатора(**allocator**) для шаблонов классов STL.

Следующий код показывает, как объявить вектор STL, который использует **cache_aligned_allocator** для выделения памяти:

```
auto values = std::vector<double,  
    cache_aligned_allocator<double>>(10000);
```

Функциональность **cache_aligned_allocator <T>** требует некоторой жертвы выделенной памяти, поскольку он должен выделить как минимум одну строку кэша памяти, даже для небольшого объекта.

Использовать **cache_aligned_allocator <T>** следует только если ложное совместное использование может стать реальной проблемой.

Потокобезопасные контейнеры oneTBB

Intel TBB предоставляет классы контейнеров (concurrent containers), которые корректно могут обновляться из нескольких потоков:

- `concurrent_vector`
- `concurrent_queue`
- `concurrent_hash_map`

Для работы в многопоточной программе со стандартными контейнерами STL доступ к ним необходимо защищать блокировками (мьютексами)

Особенности Intel TBB:

- ❑ при работе с контейнерами применяет алгоритмы не требующие блокировок (lock-free algorithms)
- ❑ при необходимости блокируются лишь небольшие участки кода контейнеров (fine-grained locking)

oneTBB concurrent_vector

concurrent_vector <T> - это динамически растущий массив **T** (аналог `std::vector`). **concurrent_vector**, позволяет безопасно увеличивать размер в то время как другие потоки также работают с его элементами или даже сами увеличивают его размер.

Для безопасного одновременного увеличения размера `concurrent_vector` имеет три метода, которые поддерживают обычное использование динамических массивов:

- ❑ **push_back(x)** - безопасно добавляет элемент **x** в контейнер
- ❑ **grow_by(n)** - безопасно добавляет **n** последовательных элементов, инициализированных с помощью `T()`.
- ❑ **grow_to_at_least(n)** - увеличивает вектор до размера **n**, если он короче. Параллельные вызовы методов роста не обязательно возвращаются в том порядке, в котором элементы добавляются к вектору.

oneTBB concurrent_vector

Пример

```
#include <tbb/concurrent_vector.h>
```

добавления текстовых данных разной длины в вектор

```
void Append(concurrent_vector<char>& vec, const char* string)
{
    size_t n = strlen(string) + 1;
    std::copy(string, string + n, vec.grow_by(n));
}
```

oneTBB concurrent_vector

Пример TBB

```
#include <tbb/concurrent_vector.h>

void Demo_Concurrent_vector_TBB()
{
    std::cout << "Демонстрация tbb::concurrent_vector с  

    tbb::parallel_for " << std::endl;

    tbb::concurrent_vector<double,  

        tbb::cache_aligned_allocator<double>> values;

    tbb::parallel_for(tbb::blocked_range<int>(0, 10000),  

        [&](tbb::blocked_range<int> r)  

        {  

            for (int i = r.begin(); i < r.end(); i++)  

            {  

                values.push_back((i * 0.01) * sin(i));  

            }  

        });  

}
```

oneTBB concurrent_vector

Пример OpenMP

```
#include <tbb/concurrent_vector.h>

void Demo_Concurrent_vector_OpenMP()
{
    std::cout << "Демонстрация tbb::concurrent_vector с  

        omp parallel for " << std::endl;

    tbb::concurrent_vector<double,  

        tbb::cache_aligned_allocator<double>> values;

    #pragma omp parallel for  

    for (int i = 0; i < 10000; i++)
    {
        values.push_back((i * 0.01) * sin(i));
    }
}
```

!!! Порядок следования элементов в векторе может быть не определен (возможно не соответствие последовательной реализации)

oneTBV concurrent_queue

Класс шаблона **concurrent_queue<T, Alloc>** реализует параллельную очередь со значениями типа **T** (аналог **std::queue**). **concurrent_vector**, позволяет безопасно увеличивать размер в то время как другие потоки также работают с его элементами или даже сами увеличивают его размер.

Основные операции над **concurrent_queue** – **push** и **try_pop**.

- ❑ **push** работает так же, как **push** для **std::queue**.
- ❑ **try_pop** выводит элемент, если он доступен. Проверка и извлечение должны выполняться за одну операцию в целях безопасности потока.

Класс шаблона **concurrent_queue** неограничен и не имеет ожидающих методов. Пользователь должен обеспечить синхронизацию, чтобы избежать переполнения, или дождаться, пока очередь не станет непустой. Обычно это уместно, когда синхронизация должна выполняться на более высоком уровне.

oneTBB concurrent_queue

Пример адаптации std::queue

```
#include <tbb/concurrent_queue.h>
```

```
//... Фрагмент последовательной версии...
```

```
std::queue<T> MySerialQueue;  
T item;  
if (!MySerialQueue.empty())  
{  
    item = MySerialQueue.front();  
    MySerialQueue.pop_front();  
    //... обработка item...  
}
```

```
//... Фрагмент concurrent_queue версии...
```

```
concurrent_queue<T> MyQueue;  
T item;  
if (MyQueue.try_pop(item))  
{  
    //... обработка item...  
}
```

oneTBV concurrent_bounded_queue

Класс шаблона **concurrent_bounded_queue** <T, Alloc> – это вариант, который добавляет операции блокировки и возможность указывать емкость. Особый интерес представляют следующие методы:

- ❑ **pop (item)** ждет, пока это не выполнится.
- ❑ **push (item)** ожидает, пока не превысит емкость очереди.
- ❑ **try_push (item)** отправляет элемент только в том случае, если он не превышает емкость очереди.
- ❑ **size ()** возвращает целое число со знаком.

Значение **concurrent_queue::size()** определяется как количество запущенных операций **push** за вычетом количества запущенных операций **pop**. Если количество **pop** превышает количество **push**, **size()** становится отрицательным.

Пример, если **concurrent_queue** пуста и есть **n** ожидающих **pop** операций, **size()** возвращает **-n**. Это предоставляет простой способ узнать, сколько «потребителей» ожидают в очереди.

Метод **empty()** определяется как истинный тогда и только тогда, когда **size()** не является положительным.

Взаимные исключения (Mutual exclusion)

Взаимные исключения (mutual exclusion) позволяют управлять количеством потоков, одновременно выполняющих заданный участок кода.

mutex – примитив синхронизации, обеспечивающий взаимное исключение исполнения критических участков кода. Задачей мьютекса является защита объекта от доступа к нему других потоков, отличных от того, который завладел мьютексом.

Классический мьютекс имеет эксклюзивного владельца, который и должен его освободить (т.е. переводить в незаблокированное состояние).

Условно классический мьютекс можно представить в виде переменной, которая может находиться в двух состояниях: в заблокированном и в незаблокированном. При входе в свою критическую секцию поток вызывает функцию перевода мьютекса в заблокированное состояние, при этом поток блокируется до освобождения мьютекса, если другой поток уже владеет им. При выходе из критической секции поток вызывает функцию перевода мьютекса в незаблокированное состояние.

oneTBB mutex

Классы **mutex** моделирует требования Mutex с использованием адаптивного подхода. При этом гарантируется, что поток, который не может получить блокировку, будет в состоянии ожидания перед блокировкой.

Виды мьютексов в **oneTBB** :

- mutex
- rw_mutex
- spin_mutex
- spin_rw_mutex
- speculative_spin_mutex
- speculative_spin_rw_mutex
- queuing_mutex
- queuing_rw_mutex
- null_mutex
- null_rw_mutex

Особенности oneTBB mutex

Выбор правильного мьютекса помогает повысить производительность. Мьютексы можно описать по следующим качествам:

- ❑ **Scalable** (Масштабируемость). Некоторые мьютексы называются масштабируемыми. В строгом смысле это не точное имя, потому что мьютекс ограничивает выполнение одним потоком за раз. Масштабируемый мьютекс не хуже этого.
- ❑ **Fair** (Справедливый) – мьютексы захватываются в порядке обращения к ним потоков (даже если следующий поток в очереди находится в состоянии сна; несправедливые мьютексы могут быть быстрее, потому что они позволяют потокам, которые выполняются, проходить первыми, а не ожидающим потокам).
- ❑ **Recursive** – мьютексы позволяют потоку захватившему мьютекс повторно его получить
- ❑ **Yield** – при длительном ожидании мьютекса поток периодически проверяет его текущее состояние и снимает себя с процессора (засыпает),
- ❑ **Block** – потока освобождает процессор до тех пор, пока не освободится мьютекс (такие мьютексы рекомендуется использовать при длительных ожиданиях)

Виды oneTBB mutex

spin_mutex – поток ожидающий освобождения мьютекса выполняет пустой цикл ожидания (**busy wait**). Его рекомендуется использовать для защиты небольших участков кода (нескольких инструкций).

mutex – имеет поведение, подобное **spin_mutex**, однако блокируется при длительном ожидании, что делает его устойчивым к высокой конкуренции.

queuing_mutex – не является рекурсивным, выполняет активное ожидание захвата мьютекса с сохранение очередности потоков, т.е. выполнение потоков продолжается в том порядке, в котором они пытались захватить мьютекс.

spin_rw_mutex и **queuing_rw_mutex** подобны **spin_mutex** и **queuing_mutex**, но дополнительно поддерживает блокировку на чтение.

rw_mutex похож на **mutex**, но дополнительно поддерживает блокировки чтения.

speculative_spin_mutex и **speculative_spin_rw_mutex** подобны **spin_mutex** и **spin_rw_mutex**, но дополнительно обеспечивают спекулятивную блокировку процессоров, поддерживающих аппаратную память транзакций. Эти мьютексы масштабируются при работе с низкой частотой конфликтов, то есть в основном в режиме спекулятивной блокировки.

Спекулятивная блокировка позволяет нескольким потокам получить одну и ту же блокировку, пока нет «конфликтов», которые могут привести к результатам, отличным от неспекулятивной блокировки.

Сравнение oneTBB mutex

Mutex	Scalable	Fair	Recursive	Long Wait	Size
spin_mutex	нет	нет	нет	yields	1 byte
mutex	да	нет	нет	blocks	1 byte
speculative_spin_mutex	HW dependent	нет	нет	yields	2 cache lines
queuing_mutex	да	да	нет	yields	1 word
spin_rw_mutex	нет	нет	нет	yields	1 word
spin_rw_mutex	да	нет	нет	blocks	1 word
speculative_spin_rw_mutex	HW dependent	нет	нет	yields	3 cache lines
queuing_rw_mutex	да	да	нет	yields	1 word

null_mutex и **null_rw_mutex** ничего не делают. Они могут быть полезны в качестве аргументов шаблона.

mutex

Пример использования

Пример добавления данных в вектор с выводом обрабатываемого диапазона на экран

```
auto values = std::vector<double>(10000);
std::mutex m;
tbb::parallel_for(tbb::blocked_range<int>(0, values.size()),
    [&](tbb::blocked_range<int> r)
    {
        m.lock();
        std::cout << "Range size " << (r.end() - r.begin())
            << " from " << r.begin() << " to " << r.end() << std::endl;
        m.unlock();
        for (int i = r.begin(); i < r.end(); ++i)
        {
            values[i] = (i * 0.01)*sin(i);
        }
    });
```


oneTBB scoped_lock

Интерфейсы реализуют шаблон блокировки с областью действия, который широко используется в библиотеках C++, поскольку:

- ❑ Не требует писать конструкции снятия блокировки
- ❑ Снимает блокировку, если генерируется исключение из области взаимного исключения, защищенной блокировкой.

Существует две части шаблона: объект-мьютекс, для которого создание объекта блокировки получает блокировку мьютекса, а деструктор объекта блокировки снимает блокировку. Пример:

```
{  
    // Создание myLock получает блокировку myMutex  
    M::scoped_lock myLock(myMutex);  
    // ... действия, которые необходимо выполнить...  
    // Уничтожение myLock снимает блокировку myMutex  
}
```

Тип **M** удовлетворяет требованиям Mutex.

Если действия вызывают исключение, блокировка автоматически снимается при выходе из блока.

oneTBB spin_mutex scoped_lock

Пример

```
#include <tbb/spin_mutex.h>
```

```
void SpinMutex_Example()
```

```
{
```

```
    constexpr int N = 100;
```

```
    tbb::spin_mutex mutex;
```

```
    auto task1 = [&mutex]() {
```

```
        for (auto i = 1; i < N; ++i)
```

```
        {
```

```
            tbb::spin_mutex::scoped_lock lock{ mutex };
```

```
            std::cout << "Task1 - " << i * i / i << std::endl;
```

```
        } };
```

```
    auto task2 = [&mutex]() {
```

```
        for (auto i = 1; i < N; ++i)
```

```
        {
```

```
            tbb::spin_mutex::scoped_lock lock{ mutex };
```

```
            std::cout << "Task2 - " << i * i / i << std::endl;
```

```
        } };
```

```
tbb::parallel_invoke(task1, task2);
```

```
}
```